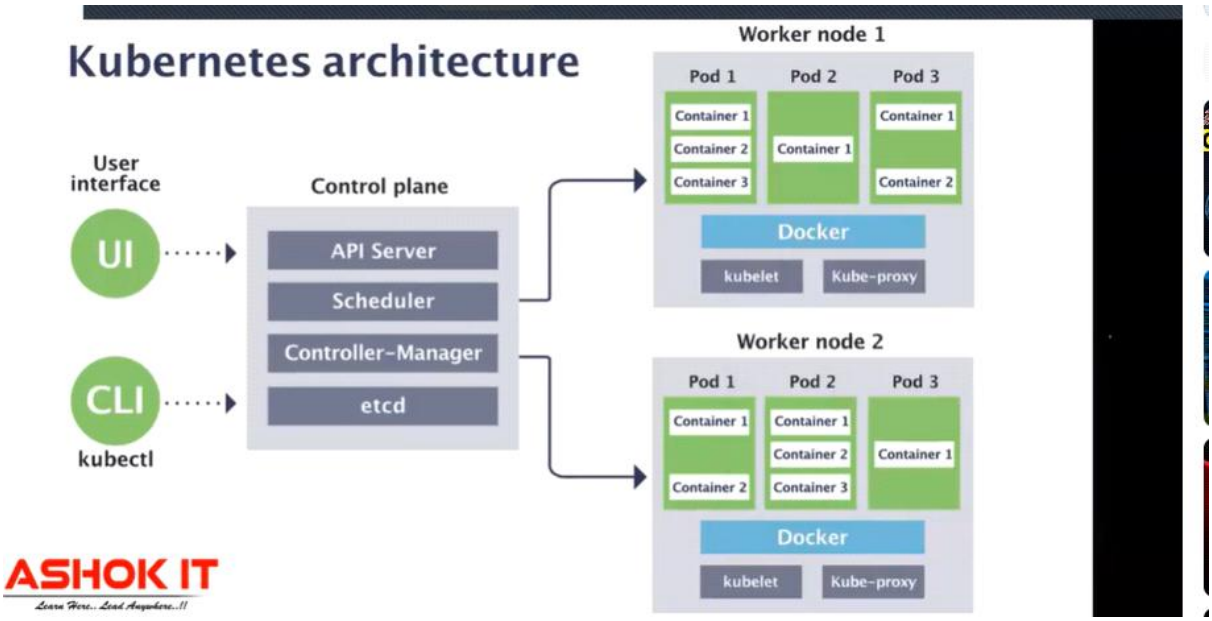
K8s-Controlollers

Monday, October 26, 2026 9:26 AM



++++++++++++++++++++++++++++++

What is Replication Controller ?

++++++++++++++++++++++++++++++

-> It is one of the key feature in k8s

-> It is responsible to manage POD lifecycle

-> It will make sure given no.of POD replicas are running at any point of time.

Note: if any POD got crashed/deleted/dead then Replication Controller will replace it.

-> Replication Controller is providing facility to create multiple PODS and it will make sure PODS always exists to run our application.

-> Using Replication controller we can achieve High Availability

-> Replication Controller and PODS are associated with Labels and Selectors.

++++++++++++++++

What is Replica Set?

++++++++++++++++

-> It is next generation of Replication Controller

-> It is also used to manage POD life cycle

-> We can scale up and scale down PODS using Replica Set also

-> The only difference between Replication Controller and Replication Set is 'Selector support'

-> We have 2 types of Selectors

1) Equality Selector

Ex:

```
selector:
  app: javawebappp
```

2) Set based Selector

Ex:

```
selector:
  matchExpressions:
    - key : app
      operator : in
            values:
              - javapp
              - javaweb
              - javawebapp
```

1. Label Selector Support

ReplicationController

Only supports **equality-based selectors**.

```yaml
selector:
  app: java-app
```

Meaning:

```
app = java-app
```

Only exact matching.

ReplicaSet

Supports:

- Equality-based selectors
- Set-based selectors

```yaml
selector:
  matchLabels:
    app: java-app
```

OR

```yaml
selector:
  matchExpressions:
  - key: app
    operator: In
    values:
    - java-app
    - web-app
```

This provides more flexibility.

✅ Interview Point: ReplicaSet supports advanced label selectors.

Visual Flow

All code copied to clipboard

```
ReplicaSet Selector

app IN (java-app, web-app)

          |
          |
   -----------------
   |       |       |
Pod-1   Pod-2   Pod-3
app=    app=    app=
java    web     payment

 ✅      ✅      ❌
```

So remember:

✅ **Replicas = Number of Pods to maintain**
✅ **Labels = Identity of Pods**
✅ **Selector = Used to find/manage Pods with matching labels**

==========

==========

DaemonSet is a Kubernetes controller that ensures **one Pod runs on every worker node.**

DaemonSet = One Pod on Every Node

Suppose you have 3 worker nodes:

```
Node-1
Node-2
Node-3
``
```

If you create a **DaemonSet**:

```
DaemonSet
    ↓
Node-1 → 1 Pod
Node-2 → 1 Pod
Node-3 → 1 Pod
```

Total:

```
3 Nodes = 3 Pods
```

New Node Added

Suppose a new node joins:

```
Node-1
Node-2
Node-3
Node-4
```

DaemonSet automatically creates:

```
Node-4 → 1 Pod
```

Now:

```
4 Nodes = 4 Pods
```

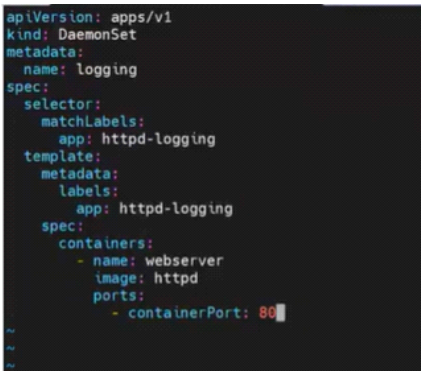
Real-Life Example

For monitoring:

```
Node-1 → Prometheus Exporter
Node-2 → Prometheus Exporter
Node-3 → Prometheus Exporter
```

Every node must be monitored.

So we use **DaemonSet**.



```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: logging
spec:
  selector:
    matchLabels:
      app: httpd-logging
  template:
    metadata:
      labels:
        app: httpd-logging
    spec:
      containers:
        - name: webserver
          image: httpd
          ports:
            - containerPort: 80
```

Manually POD Created (Not recommended)

POD creation using ReplicationController

POD creation using ReplicaSet

POD creation using DaemonSet

-> In above concepts Auto-Scaling feature not available (Manuallu we need to scale our pods)

-> There is no option to rollback our pods creation.

=> To overcome these problems We have "Deployment" concept

++

What is Deployment ?
++++++++++++++++++++

-> Deploymet is used to tell Kubernetes how to create or modify instances of the pods

-> By using Deployment we can rollout and rollback our application deployment (if required)

-> We can achieve Auto-Scaling by Deployment

++++++++++++++++++++
Deployment Strategy
++++++++++++++++++++

1) ReCreate

2) Rolling Update

3) Blue / Green (Approach)

Newly added File why it is not added